Supplementary Material

to "A Fail-Safe String-Level Reconfigurable Battery Management System Using Discrete Back-to-Back P-MOSFET Switches: Design and Rover Validation on a 3S2P Li-Ion Pack"

This document gives the detailed test tables, the full failure analysis, the pin allocation and sample firmware code that support the paper. Section and table numbers of the paper are cited as "paper" to avoid confusion with the numbering used here. As in the paper, every result is labelled by its evidence level: simulated results come from running the unmodified firmware modules on a computer against a plant model, and are not hardware measurements.

S1. Host-Simulation Scenarios

The safety and reconfiguration modules were compiled with gcc (C99, -Wall -Wextra -Wpedantic -Wconversion -Wshadow) without modification and run against a plant model with open-circuit voltage and internal resistance per string, a 30 mΩ switch pair, a current-sink load, injected faults and ±3 mV / ±5 mA measurement noise (paper, Section V). Table S1 groups the 22 safety-layer scenarios by behaviour. Table S2 lists the 18 reconfiguration scenarios; P5a and P5b share one row.

Table S1: Safety-layer host-simulation scenarios (simulated)

Scenario	Result
Boot hold-off	All switches blocked until the third clean sample
Injected over-current on B while A runs	B trips on the third sample (60 ms); A and the load keep running; label shows the fault as injected
Faulted string re-closed by other code	Forced off in the next cycle
Two 12.5 A samples, then a normal one	No trip; three consecutive samples trip
Low voltage on a freshly connected string	UV latched 140 ms after the low reading (100 ms settle + 3 samples)
0 V on all-off strings for 20 samples	No fault (no start-up deadlock)
600 mA through an OFF string	STUCK; all switches off including the load
Current within 100 ms of an OFF command	Not treated as stuck
Sensor B unreadable	All off in the same cycle; everything blocked
−4.5 A charge current; 12.65 V	OCC and OV latched
Sensor missing at start-up	Everything blocked

Table S2: Reconfiguration host-simulation scenarios (simulated)

ID	Scenario and outcome
P1	Start-up and join at rest; true ΔV at the join 37.5 mV
P2	Parallel demonstration: B opens, load never interrupted, motors keep running, A carries the load
P3	ΔV of 150 mV: never joins
P4	0.8 A load pulls A into the gate window; joins while driving (true ΔV 68.7 mV)
P4b	2 A load: no join while driving; joins at rest
P5a / P5b	95 mV joins; 105 mV refused
P6	Failed-open B dropped and never rejoined
P8	Switch turned on by other code: LOCKOUT
P9	Sensor lost: LOCKOUT
P10	Both strings trip: LOCKOUT
P11	Single-string swap in the parallel build
S1	Single-string build: start-up on B
S2	Single-string build: demonstration swap from B to A with break-before-make
S3	Fault with motors stopped: LOCKOUT (open cannot be proven)
S4	Stuck B: STUCK, then LOCKOUT; A never closed
S6	Emergency stop command: LOCKOUT
S7	Idle string faults (ignored), then both down: LOCKOUT

P1 to P11 (12 scenarios, counting P5a and P5b separately) use the parallel build; S1 to S7 (6 scenarios) use the single-string build. All 18 passed.

S2. Mutation Tests

Deliberate errors were introduced into the reconfiguration module one at a time to check that the test harness detects them (Table S3). Six of the seven single mutants were detected. The undetected mutant removed one of two redundant block-mask checks; every caller performs the first check, and the mutant that removed both checks was detected.

Table S3: Mutation tests on the reconfiguration module (simulated)

Deliberate error	**Detected?**
ΔV gate raised from 100 mV to 300 mV	Yes
Gate on ΔV_{lo} only (load term removed)	Yes
No cooldown before a join	Yes
Swap without the ‘current flowed before’ proof	Yes
Swap without waiting for open confirmation	Yes
Motors stopped before the open proof	Yes
Block-mask check removed inside the close routine only	No (redundant check)
Block-mask check removed in both places	Yes

S3. Failure Modes and Effects Analysis

Table S4 is the full analysis condensed in paper Table IV. It follows the structure of ISO 26262 and IEC 61508 analyses (what fails, how it is detected, the safe response) without any claim of compliance.

Table S4: Failure modes and effects analysis of the switching hardware and control code

Failure	**Detection**	**Safe response**
Close onto a string at a large ΔV	ΔV gate on both bounds, five samples	Join refused; stay on one string
Sensor reads the bus instead of its string	Not detectable in firmware	Paralleling compiled out unless confirmed on the bench
Pair stuck closed during a swap	Open never confirmed; STUCK at about 160 ms	LOCKOUT; new string never closed
Open cannot be proven (no load before the trip)	Current before the trip < 300 mA	LOCKOUT
Joined pair fails open or is intermittent	< 100 mA while the total load ≥ 1.5 A	Open it; no rejoin until reset
Other code moves a switch	Command differs from output register	LOCKOUT
NPN shorted collector to emitter	Current in an OFF string	STUCK: all off including load
NPN open	Pair does not conduct when commanded	String unavailable; other string used
MOSFET gate oxide punctured	Readings contradict the commanded state	All off
Sensor lost or bus held low	Timeout; recovery fails	Sensor fault: all off, latch
Loop hangs	Watchdog timeout	Reset into hardware all-off
HAL fault	Error handler entered	Direct register write: all off
Pair shorted before power-up, no load	Not detectable	Fuse; bench procedure
Load-switch inrush into the driver’s input capacitance	Not visible at 20 ms sampling	Fuse; precharge in the next revision

S4. Pin Allocation

Table S5 lists the STM32F446RE pin allocation, checked against the netlist exported from the KiCad schematic of the fabricated board. The switch channel itself is shown in paper Fig. 3.

Table S5: STM32F446RE pin allocation (verified against the PCB netlist)

Function	MCU pin	Connection
String A switch	PB4	D1 → R1 → base of Q3; Q3 collector on net Gate_Q1 (Q1, Q2)
String B switch	PB5	D2 → R4 → base of Q4; Q4 collector on net Gate_Q5 (Q5, Q6)
Load switch	PB10	D3 → R7 → base of Q7; Q7 collector on net Gate_Q8 (Q8, Q9)
I2C1 SCL / SDA	PB8 / PB9	INA260 A (U2) and OLED (DS1)
I2C3 SCL / SDA	PA8 / PC9	INA260 B (U3)
Motor IN1 / IN2	PA9 / PA10	L298N module (U4) pins 7, 8
Motor IN3 / IN4	PB3 / PC7	L298N module (U4) pins 9, 10
Telemetry UART	PA2 / PA3	USART2 to the on-board ST-Link virtual COM port
Drive button	PC13	On-board user button
Heartbeat LED	PA5	On-board LED
Board supply	VIN (CN6 pin 8)	Net STR_A, upstream of the string A switch

S5. Sample Firmware Code

Listing S1 shows the debounce function, the per-string checks and the enforcement step of the safety layer. Listing S2 shows the error handler, which forces all three switch pins low with a single register write before disabling interrupts. The reconfiguration module is not reproduced.

Listing S1: Debounce, checks and enforcement in bms_safety.c (excerpt)

```
/* One debounce step. Returns 1 when the check has been bad for N
   consecutive samples. Any good sample resets the count to 0. */
static uint8_t debounce(uint8_t *cnt, uint8_t bad)
{
    if (!bad) { *cnt = 0u; return 0u; }
    if (*cnt < 255u) (*cnt)++;
    return (*cnt >= SAFETY_DEBOUNCE_N) ? 1u : 0u;
}

        uint8_t bad[CHK_COUNT];
        bad[CHK_OV]    = (v_trust && v_mV > SAFETY_OV_MV) ? 1u : 0u;
        bad[CHK_UV]    = (v_trust && v_mV < SAFETY_UV_MV) ? 1u : 0u;
        bad[CHK_OCD]   = ( i_dsg > SAFETY_OC_DSG_MA) ? 1u : 0u;
        bad[CHK_OCC]   = (-i_dsg > SAFETY_OC_CHG_MA) ? 1u : 0u;
        /* Stuck: judged on the REAL current, never the injected one, and
           only once the OFF command has had time to take effect. */
        bad[CHK_STUCK] = (!on
                          && (uint32_t)(now - t_off[s]) >= SAFETY_STUCK_SETTLE_MS
                          && iabs(i_real) > SAFETY_STUCK_MA) ? 1u : 0u;

    /* 6. Enforce, every call. */
    if (any_global())
    {
        Switch_AllOff();                    /* strings AND load switch */
    }
    else
    {
        for (uint8_t s = 0; s < BMS_NUM_STR; s++)
            if (str_bits(s)) Switch_Set(sw_of[s], SW_OFF);
    }
```

Listing S2: Error handler forcing the safe state

```
void Error_Handler(void)
{
    /* Safe state FIRST, before interrupts go away and before anything
       that depends on HAL - HAL may be the failed subsystem.
       Upper 16 bits of BSRR = atomic reset. */
    GPIOB->BSRR = (uint32_t)(GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_10) << 16u;
    __disable_irq();
    while (1) { }
}
```